

## Roadmap: Perito Judicial em Computação Forense

Este roadmap foi desenhado para construir uma base sólida em infraestrutura (Sistemas Operacionais) e linguagens essenciais para análise de artefatos, engenharia reversa e automação pericial.

### Dúvida Inicial: Qual a diferença entre C e C++?

Antes de entrar no roadmap, é crucial entender a diferença entre as duas:

**C (Procedural):** É uma linguagem mais antiga, puramente procedural (baseada em funções). Ela não possui o conceito de "objetos" ou "classes". É extremamente leve e próxima do hardware. O kernel do Windows e do Linux são escritos majoritariamente em C. Na perícia, você usa o conhecimento em C para ler a memória física, entender como o sistema operacional gerencia processos e analisar artefatos brutos (bytes).

**C++ (Orientada a Objetos):** Nasceu como uma extensão do C ("C com classes"). Ela suporta tudo que o C faz, mas adiciona Programação Orientada a Objetos (classes, herança, polimorfismo). Softwares mais complexos (como navegadores, motores de jogos e muitos malwares modernos) são escritos em C++ para organizar melhor o código. Na perícia, você usa C++ para fazer engenharia reversa de malwares complexos e entender como eles manipulam objetos no Windows.

### 1. Sistemas Operacionais (A Base da Perícia)

*O que é:* O terreno onde os crimes digitais acontecem. Você não investiga a linguagem, investiga o impacto dela no SO.

**Endpoint (Ponto de Chegada):** Ser capaz de olhar para um disco rígido bruto (imagem forense) ou sistema em execução e extrair a "linha do tempo" exata do que o usuário ou invasor fez, mesmo que ele tenha tentado apagar.

#### 1.1. Windows Internals (Do que é feito e como)

**Arquitetura:** Divisão entre *User Mode* (onde rodam os apps) e *Kernel Mode* (o núcleo do sistema). Entender como os processos se comunicam via chamadas de sistema (API do Windows / NTAPI).

**Sistema de Arquivos (NTFS):** Como o Windows guarda arquivos físicos. Estudo da **MFT** (Master File Table - o "índice" do disco), *Alternate Data Streams* (ADS - onde malwares se escondem) e recuperação de arquivos deletados (*File Carving*).

**Artefatos de Execução:** Provar que algo rodou na máquina. Estudo profundo do Registro do Windows (onde ficam as configurações), *Prefetch*, *Amcache* e *Shimcache* (rastros de programas executados).

**Event Logs (EVTX):** O diário do Windows. Rastrear logins, falhas de segurança e serviços criados pelo invasor.

## 1.2. Linux Internals (Do que é feito e como)

- **Arquitetura e Hierarquia:** O conceito de que "tudo no Linux é um arquivo". Estrutura de diretórios (/etc, /var, /home, /dev).
- **Sistema de Arquivos (EXT4 / XFS):** Como o Linux mapeia dados. Entender os *Inodes* (metadados dos arquivos que contêm datas de criação/modificação e permissões).
- **Gerenciamento de Processos e Serviços:** Como rastrear processos maliciosos escondidos usando o diretório /proc e analisando serviços via systemd.
- **Artefatos e Logs:** Rastrear acessos e comandos executados analisando o /var/log (*syslog*, *auth.log*) e o histórico de comandos (.bash\_history).

## 2. Linguagens C e C++ (O Baixo Nível)

*O que é:* As ferramentas para olhar debaixo do capô dos binários, vírus e memória RAM.

- **Endpoint (Ponto de Chegada):** Ser capaz de descompilar um executável malicioso, entender suas funções na memória (Engenharia Reversa) e criar *PoCs* (Provas de Conceito) provando para o juiz como um ataque de buffer overflow funciona.

### 2.1. Elementos Menores (Do que é feito e como)

- **Gerenciamento de Memória (C/C++):** Aprender o que são ponteiros (variáveis que guardam endereços de memória). Essencial para extrair chaves de criptografia diretamente de despejos de memória RAM (*Memory Forensics*).
- **Estruturas e Manipulação de Bits (C):** Usar structs para mapear cabeçalhos de arquivos corrompidos (ex: recriar o cabeçalho de um PDF ou JPEG destruído byte a byte).
- **Chamadas de API do SO (C/C++):** Entender como um malware escrito em C chama as funções do Windows (ex: CreateProcess, VirtualAlloc) para se injetar em outro programa legítimo.

## 3. Scripting: Bash e PowerShell (Automação e Auditoria)

*O que é:* O idioma nativo dos terminais dos sistemas operacionais. Usado por administradores, mas muito abusado por atacantes (Técnicas *Living off the Land*).

- **Endpoint (Ponto de Chegada):** Ser capaz de criar scripts para coletar evidências em tempo real (Live Response) sem desligar a máquina suspeita, e conseguir "desofuscar" scripts maliciosos usados em ataques.

### 3.1. PowerShell - Windows (Do que é feito e como)

- **Orientação a Objetos:** Diferente de outros terminais que cospem texto, o PowerShell retorna objetos. Entender os *Cmdlets* e a estrutura de *Pipeline* (|).
- **Integração Profunda:** Como o PowerShell interage com o WMI (Windows Management Instrumentation) e o Active Directory.

- **Forense Ofensiva/Defensiva:** Analisar logs de bloqueio de script (*Script Block Logging*), reverter ofuscação (quando o invasor embaralha o código) e criar rotinas de coleta de Registro.

### 3.2. Bash - Linux (Do que é feito e como)

- **Ferramentas Canivete-Suíço:** Dominar utilitários de texto como grep (buscar padrões), awk e sed (manipular colunas e strings) para filtrar milhares de linhas de log em segundos.
- **Automação e Agendamento:** Entender variáveis de ambiente, fluxos de repetição (loops) e como auditar *Cronjobs* (tarefas agendadas que hackers usam para manter persistência no servidor).

### 4. Go / Golang (Triagem e Processamento de Dados)

*O que é:* A linguagem moderna, rápida e portátil, ideal para lidar com a quantidade absurda de dados gerada em incidentes corporativos.

- **Endpoint (Ponto de Chegada):** Desenvolver suas próprias ferramentas forenses que podem ser compiladas em um único arquivo (sem precisar instalar nada na máquina suspeita) e que consigam processar terabytes de logs simultaneamente.

#### 4.1. Elementos Menores (Do que é feito e como)

- **Compilação Estática e Cross-Platform:** Como escrever o código uma vez no seu Linux de análise e compilar um .exe perfeito para rodar no servidor Windows que foi invadido.
- **Concorrência (Goroutines e Channels):** O "superpoder" do Go. Como criar scripts que varrem centenas de milhares de arquivos de log do servidor web simultaneamente (usando todos os núcleos do processador) sem travar a máquina.
- **Interação com Redes/APIs:** Construir parsers rápidos para analisar tráfego de rede capturado (arquivos .pcap) ou cruzar IPs encontrados em logs com bases de inteligência de ameaças (Threat Intelligence) de forma automatizada.